

Recursively Solving the Sliding-Tile Puzzle

Now that we have the functions for creating and manipulating the puzzle data structure, let's create the functions that solve the puzzle by recursively sliding the tiles in each possible direction and checking whether this produces a finished, ordered board.

The `attemptMove()` function performs a single slide on a board data structure, then recursively calls itself once for each of the valid moves the board can make. Multiple base cases exist. If the board data structure is in a solved state, the function returns a Boolean `True` value; if the maximum number of moves has been reached, it returns a Boolean `False` value. Also, if a recursive call has returned `True`, then `attemptMove()` should return `True`, and if recursive calls for all the valid moves have returned `False`, then `attemptMove()` should return `False`.

The solve() Function

The `solve()` function takes a board data structure and maximum number of moves the algorithm should attempt before backtracking. Then it performs the first call to `attemptMove()`. If this first call to `attemptMove()` returns `True`, the code in `solve()` displays the series of steps that solves the puzzle. If it returns `False`, the code in `solve()` tells the user no solution was found with this maximum number of moves.

The Python code for `solve()` begins as follows:

Python

```
def solve(board, maxMoves):
    """Attempt to solve the puzzle in `board` in at most `maxMoves`
    moves. Returns True if solved, otherwise False."""
    print('Attempting to solve in at most', maxMoves, 'moves...')
    solutionMoves = [] # A list of UP, DOWN, LEFT, RIGHT values.
    solved = attemptMove(board, solutionMoves, maxMoves, None)
```

The JavaScript code for `solve()` begins as follows: